- **Header & Document Information**

- **Course**: CSAI 203 - Intro to Software Engineering | **Term**: Fall 2026
- Students : Eyad Elsayed (202500169) ,Ahmed Nafea (202500650) , Salma Youssef (202507819) , Nouran Osama (202509875)
- **Title**: Online Quiz & Auto-Grading System | **Deliverable**: Phase 2 SRS
-
- **Institution**: Zewail City (CSAI) | **Rep**: Eyad Elsayed (s-eyad.araby@zewailcity.edu.eg)
-
- **Section 1: Introduction**
- **1.1 Purpose**: Defines SRS functional and quality requirements to guide system development.
-
- **1.2 Scope**: Web app for question banking, timed test execution, auto-grading (MCQ/TF), result reporting, and role administration. AI training and hardware integrations excluded.
-
- **1.3 Acronyms**: SRS, SDLC, MVC, MCQ, UI/GUI, CRUD, HTTPS, CSV.
-
- **1.4 References**: 1. Dr. Mohamed Sami Rakha, *CSAI 203 Guidelines*, 2026. 2. IEEE Std 830-1998.
-
- **1.5 Overview**: Sec 1 (Intro), Sec 2 (Overall Description), Sec 3 (Requirements & Use Cases), Sec 4 (Appendices).
-
- **Section 3.1: Specific Functional Requirements (FR1 – FR5)**
- **FR1 (Login UI)**: Authenticates users (Student, Instructor, Admin) via email/password. *Primary*: Valid login opens dashboard. *Alt*: Invalid credentials display inline error.
-
- **FR2 (Question Creation UI)**: Web form for MCQ/TF questions. *Primary*: Saves stem, options, key, and points. *Alt*: Incomplete fields block save.

-
- **FR3 (Question Bank & Search UI)**: Searchable list to edit/delete items. *Primary*: Search by keyword to modify or delete items. *Alt*: Deleting active test items shows warning.
-
- **FR4 (Question Tagging)**: Metadata tag assignment. *Primary*: Attach tags (Topic/Difficulty) for sorting. *Alt*: Filter question bank by selected tags.
-
- **FR5 (Profile & Password UI)**: Account viewing and password management. *Primary*: Valid current password saves new hash. *Alt*: Incorrect password displays error.
-
- **Section 3.2.2: Use Case IEEE Description Templates (UC1 – UC5)**
- **UC1 (Login)**: *Actors*: All Roles | *Pre*: DB account exists. | *Flow*: 1. Enter email/password. 2. System validates & loads dashboard. | *Ext*: 2a. Invalid login displays error.
-
- **UC2 (Create Question)**: *Actors*: Instructor | *Pre*: Logged in. | *Flow*: 1. Input text, options, key, points. 2. Save entity. | *Ext*: 1a. Blank fields alert user.
-
- **UC3 (Edit Question)**: *Actors*: Instructor | *Pre*: Question exists. | *Flow*: 1. Select question. 2. Modify attributes and save updates.
-
- **UC4 (Tag Question)**: *Actors*: Instructor | *Pre*: Question form open. | *Flow*: 1. Enter tags. 2. Save question to attach metadata.
-
- **UC5 (Update Password)**: *Actors*: All Roles | *Pre*: Authenticated. | *Flow*: 1. Open Settings. 2. Input current/new password. 3. System validates & saves hash.
-
- **Section 2: Overall Description**

- **2.1 Product Perspective**: Web-based MVC app (View: Dashboards, Controller: Logic/Grading, Model: Relational DB).
-
- **2.2 Product Functions**: Role Management, Question Bank, Quiz Engine, Timed Execution & Auto-Grading, Gradebook & CSV Exports.
-
- **2.3 User Classes**: Admin (system accounts), Instructor (question banks/quizzes/grading), Student (takes tests/views scores).
-
- **2.4 Stakeholders**: Students (timing/scoring), Instructors (auto-grading efficiency), Evaluators (IEEE compliance), IT Dept (security/stability).
-
- **2.5 Operating Environment**: Server (2 GHz CPU, 4 GB RAM, 20 GB disk), OS (Linux/Windows, Node.js/Python, PostgreSQL/MySQL), Client (Chrome/Firefox/Edge v100+).
-
- **2.6 Constraints**: Strict MVC architecture, manual UI coding, no custom AI models, academic semester schedule.
-
- **2.7 User Documentation**: Integrated navbar Help/FAQ section.
-
- **2.8 Assumptions & Dependencies**: Internet/browser availability; active DB connection and server clock synchronization required.
-
- **Section 3.1: Specific Functional Requirements (FR6 – FR10)**
- **FR6 (Quiz Creation UI)**: Form for title, points, duration limit. *Primary*: Saves draft if duration > 0. *Alt*: Invalid inputs block submission.
-
- **FR7 (Question Selection UI)**: Link bank questions to quiz draft. *Primary*: Checking items links them and updates total points. *Alt*: Unchecking items recalculates total score.

-
- **FR8 (Basic Scheduling)**: Quiz availability start/end times. *Primary*: Validates Start < End and updates schedule. *Alt*: Invalid time range shows error.
-
- **FR9 (Access Control)**: Passcode protection setting. *Primary*: Prompts for passcode to launch test. *Alt*: Incorrect code denies access.
-
- **FR10 (Publish Toggle)**: Switch state between Draft and Published. *Primary*: Publishing renders test visible at start time. *Alt*: Unpublishing hides test.
-
- **Section 3.2.2: Use Case IEEE Description Templates (UC6 – UC10)**
- **UC6 (Create Quiz)**: *Actors*: Instructor | *Pre*: Logged in. | *Flow*: 1. Input title/duration. 2. Save draft record. | *Ext*: 1a. Missing inputs trigger error.
-
- **UC7 (Attach Questions)**: *Actors*: Instructor | *Pre*: Draft exists. | *Flow*: 1. Select bank items via checkboxes. 2. Link items and update score. | *Ext*: 1a. Empty bank alerts user.
-
- **UC8 (Schedule Quiz)**: *Actors*: Instructor | *Pre*: Draft exists. | *Flow*: 1. Enter Start/End timestamps. 2. System validates & saves. | *Ext*: 2a. End <= Start fails.
-
- **UC9 (Set Passcode)**: *Actors*: Instructor | *Pre*: Draft exists. | *Flow*: 1. Input passcode. 2. Save access rule. | *Ext*: 1a. Blank field removes passcode.
-
- **UC10 (Publish Quiz)**: *Actors*: Instructor | *Pre*: Questions attached & schedule valid. | *Flow*: 1. Toggle status to Published. 2. DB updates state. | *Ext*: 1a. Zero questions blocks publish.
-

3.4 Non-Functional Requirements (NFRs)

ID	Quality Attribute	Requirement Description	Testing Method	Success Criteria
NFR1	Performance	The application shall process quiz page loads and auto-grading operations within acceptable response limits.	**Automated Load Testing:** Simulate 50 concurrent student HTTP requests using Apache JMeter.	95% of HTTP GET/POST responses return within 2.0 seconds or less.
NFR2	Security	User passwords must be hashed prior to database storage, and unauthorized page access restricted via role check.	**Access Control Testing:** Send direct HTTP requests to protected /instructor endpoints from an unauthenticated browser session.	100% of unauthorized requests are redirected to /login with zero raw credential exposure.
NFR3	Usability	User interfaces shall render responsively without layout distortion across standard display sizes.	**Responsive Layout Testing:** Test page layouts across viewport widths from 360px (mobile) to 1920px (desktop) using developer tools.	Zero horizontal scrollbars or overlapping text elements across tested viewports.
NFR4	Reliability	The system shall handle transient network drops during quiz execution without	**Network Disconnect Injection:** Disable network connection during an active quiz session, select	Draft selections are saved locally and synced to the database within 3 seconds

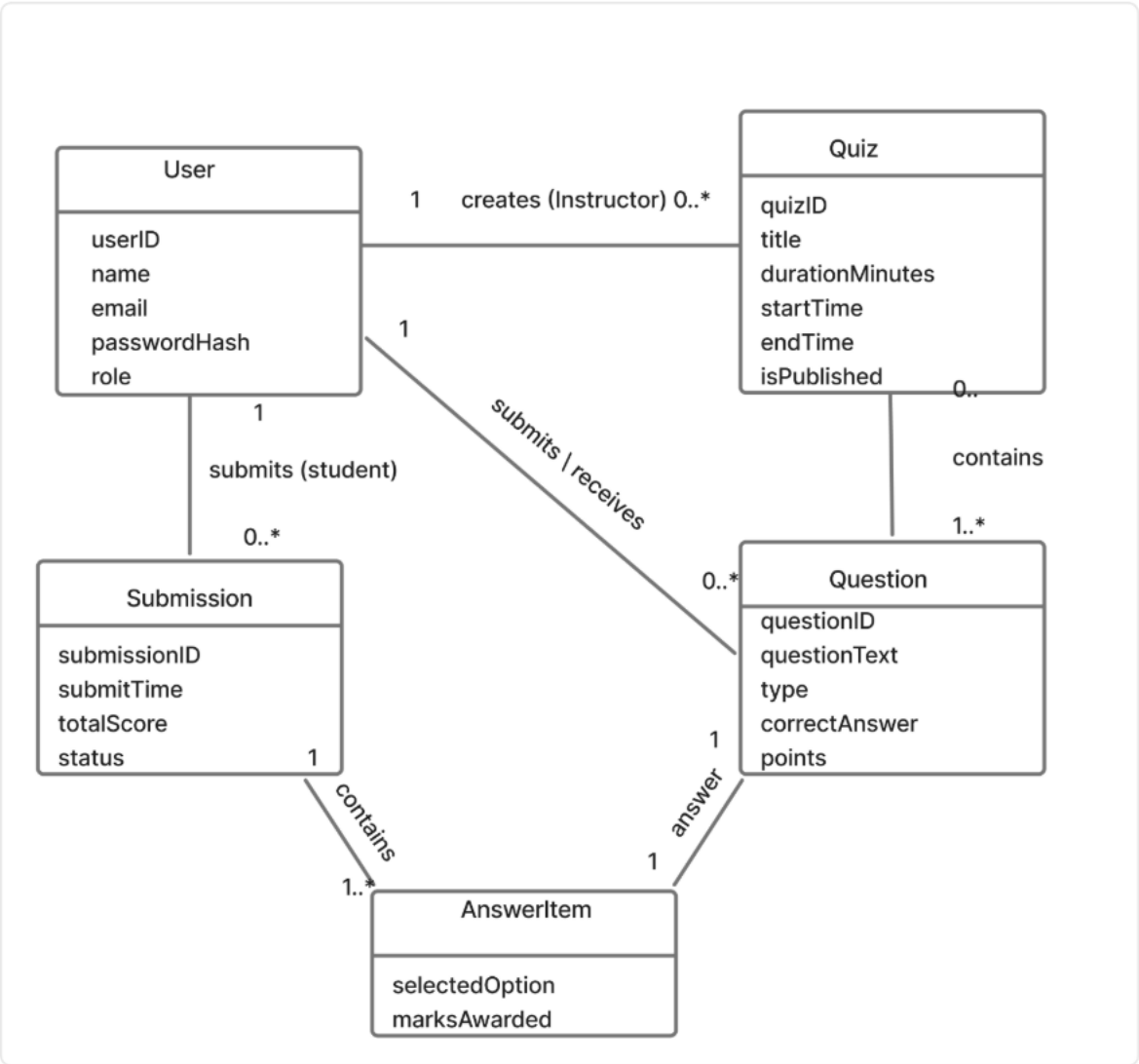
		losing selected answers.	choices, and restore connectivity.	of connection recovery.
NFR5	Compatibility	Web interfaces shall execute consistently across modern desktop web browsers.	**Cross-Browser Automated Testing:** Run UI automated test suites on Chrome, Firefox, and Edge engines.	All automated functional test cases pass with a 100% success rate across all target browsers.

Salma Youssef's Section

- **Section 3.3: Domain Model**
- ***3.3.1 Conceptual Class Diagram Overview***
- Core entities and key attributes:
-
- **User**: UserID, Name, Email, PasswordHash, Role (Student/Instructor)
-
- **Quiz**: QuizID, Title, StartTime, EndTime, DurationTime
-
- **Question**: QuestionID, QuestionText, Points, CorrectAnswer, Type (MCQ/TF)
-
- **Submission**: SubmissionID, SubmitTime, TotalScore, Status
-
- **AnswerItem**: SelectedOption, MarksAwarded
-
- **Multiplicities**:
-
- Instructor creates 0..* Quizzes (1 Quiz : 1 Instructor)
-
- Student submits $0..*$ Submissions (1 Submission : 1 Student)
-

- Quiz receives $0..*$ Submissions (1 Submission : 1 Quiz)
-
- Submission contains 1..* AnswerItems (1 AnswerItem : 1 Question)
-

Conceptual Class Diagram



3.3.2 Class Descriptions

- **User Class**: Represents system users and stores their account credentials, personal details, and assigned role.

- **Question Class**: Represents reusable test questions stored within the question bank.

- **Quiz Class**: Represents assessment configurations created by instructors, including schedule windows and publication states.

- **Submission Class**: Represents a student's attempt to complete a quiz, tracking completion status and calculated final score.

- **AnswerItem Class**: Represents an individual response to a question within a submission, capturing selected options and awarded marks.

Section 3.1: Specific Functional Requirements (FR11 – FR15)

- **FR11: Student Quiz Taking UI**

    - *Description*: The system shall provide an interactive exam interface displaying questions, choice controls, progress indicators, and a live countdown timer.

    - *Primary Scenario*: Student starts an active quiz; system renders questions with selection radio buttons, progress bar, and active countdown timer.

    - *Alternative Scenario (Draft Local Save)*: Selecting an answer saves a temporary local draft to restore choices if the browser refreshes.

- **FR12: Quiz Submission UI**

    - *Description*: The system shall render a pre-submission review screen displaying counts of answered and unanswered questions before final confirmation.

    - *Primary Scenario*: Student clicks "Finish Quiz", reviews completion summary (e.g., "Answered 9 of 10"), and clicks "Confirm Submit" to save.

- *Alternative Scenario (Cancellation)*: Student selects "Return to Quiz" to return to the exam interface and modify answers.

- **FR13: Automated Grading Engine**

  - *Description*: The system shall automatically evaluate submitted answers by comparing student responses against stored answer keys.

  - *Primary Scenario*: Upon submission, backend evaluates each AnswerItem, compares against correct keys, calculates total points, and updates Submission.totalScore.

  - *Alternative Scenario (Incorrect Answer)*: Incorrect choices award zero points for that item.

- **FR14: Instructor Grade Adjustment UI**

  - *Description*: The system shall provide an interface allowing instructors to review student submissions and manually adjust item scores.

  - *Primary Scenario*: Instructor opens submission details, modifies marks awarded for a question, and saves; system recalculates total score.

  - *Alternative Scenario (Invalid Input)*: Entering a score higher than maximum question points blocks submission and shows an error message.

- **FR15: Automatic Late Submission Block**

  - *Description*: The system shall enforce quiz time limits by locking the interface and auto-submitting answers when the countdown reaches zero.

- *Primary Scenario*: Countdown hits zero; system disables inputs, auto-submits current responses, and redirects student to completion page.

- *Alternative Scenario (Network Delay)*: Network loss at expiration preserves local drafts and dispatches them upon reconnection, marking submission as late.

Section 3.2.2: Use Case IEEE Description Templates (UC11 – UC15)

- **UC11: Take Quiz**

  - *Actors*: Student | *Preconditions*: Student logged in; quiz published and currently active.

  - *Postconditions*: Quiz attempt active and responses captured.

  - *Main Flow*: 1. Select available quiz from dashboard. 2. System verifies availability. 3. System renders questions and starts countdown timer. 4. Student selects answers and progress updates.

  - *Extensions*: 4a. Browser refresh: system restores previously selected answers from local draft.

- **UC12: Submit Quiz Answers**

  - *Actors*: Student | *Preconditions*: Active quiz attempt in progress.

  - *Postconditions*: Submission recorded in database.

  - *Main Flow*: 1. Click "Finish Quiz". 2. System displays answered/unanswered summary. 3. Click "Confirm Submit". 4. System saves submission in database and triggers grading.

- *Extensions*: 3a. Click "Return to Quiz": returns student to exam interface.

- **UC13: Automatically Grade Submission**

  - *Actors*: System / Auto-Grading Engine | *Preconditions*: Submission received; correct keys and point values stored in DB.

  - *Postconditions*: Total score calculated and stored.

  - *Main Flow*: 1. System receives submission. 2. Processes AnswerItem records. 3. Compares selections with stored keys. 4. Awards points or zero. 5. Updates Submission.totalScore.

  - *Extensions*: 4a. Incorrect choice: awards zero points for question and continues grading remaining items.

- **UC14: Override Student Grade**

  - *Actors*: Instructor | *Preconditions*: Instructor logged in; student submission exists.

  - *Postconditions*: Adjusted marks and recalculated total score saved.

  - *Main Flow*: 1. Open student submission details. 2. Select question for adjustment. 3. Input corrected mark. 4. Click "Update Score". 5. System validates mark, recalculates total, and saves update.

  - *Extensions*: 3a. Invalid mark entered (> max points): system rejects change with an error message.

- **UC15: Enforce Timer Expiration**

- *Actors*: System / Timer Service | *Preconditions*: Active quiz attempt; countdown running.

- *Postconditions*: Interface locked; current answers auto-submitted.

- *Main Flow*: 1. Countdown timer hits zero. 2. System disables inputs. 3. System auto-submits current responses. 4. System redirects to completion page.

- *Extensions*: 3a. Connection lost at 00:00: system retains local draft. 3b. Connection restored: system dispatches saved draft and flags as late.

Section 4.1: Appendix A — Data Dictionary

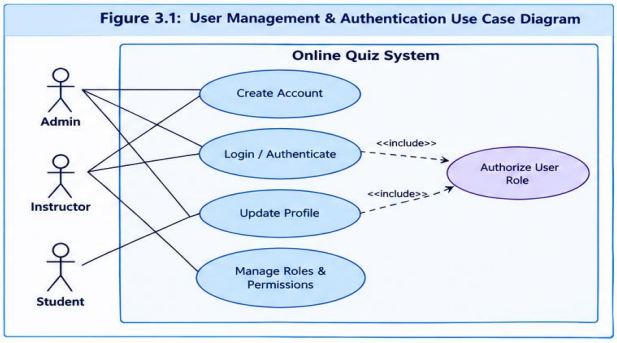
Table Name	**Attribute**	**Data Type**	**Description**
User	User_id	Primary key (INT)	Unique identifier for registered user.
User	Email	VARCHAR (100)	User login email address
User	Password_hash	VARCHAR (225)	Encrypted user password string
User	role	VARCHAR (20)	User role ('Admin', 'Instructor', 'Student').

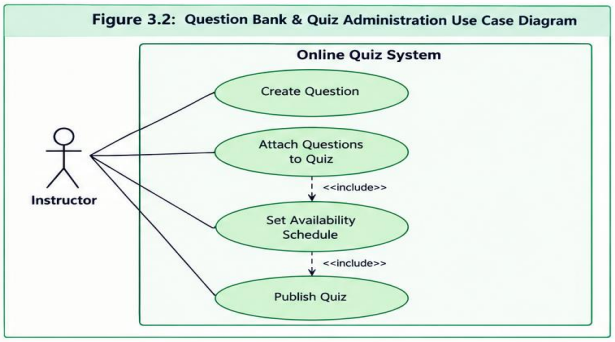
Questions	question_id	Primary Key (INT)	Unique question item identifier.
Questions	question_text	TEXT	Detailed stem text of the question.
Questions	question_text	QuestionTEXT	Detailed stem text of the question.
Quizzes	quiz_id	Primary Key (INT)	Unique identifier for created quiz.
Quizzes	duration_minutes	INT	Assigned time limit in minutes.
submissions	submission_id	Primary Key (INT)	Unique submission attempt identifier.
Submissions	total_score	FLOAT	Computed grade score earned by student.

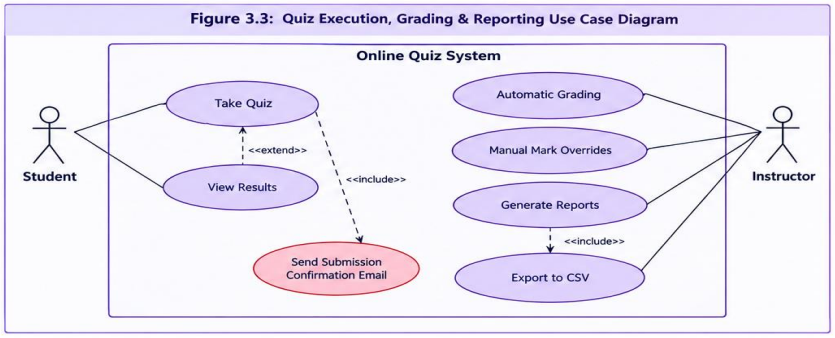
1. **Nouran Osama's Section**

2. **Section 3.2.1: Use Case Diagrams Overview**
3. The system use cases are organized into three distinct UML diagrams to represent user interactions across system boundaries:
4. **User Management & Authentication Use Case Diagram**: Covers user login, profile updates, account creation, and role authorization for Admin, Instructor, and Student actors.
5. **Question Bank & Quiz Administration Use Case Diagram**: Covers creating questions, attaching questions to quizzes, setting availability schedules, and publishing quizzes by the Instructor actor.

6. **Quiz Execution, Grading & Reporting Use Case Diagram**: Covers taking quizzes and viewing results for the Student actor, alongside automated grading, manual mark overrides, gradebook analytics, and CSV exports for the Instructor actor.







- **Section 3.5: External Interface Requirements**
- **3.5.1 UI (HTML5/CSS3/Bootstrap)**: Login (credentials/role), Student Dashboard (quizzes/history/grades), Quiz View (questions/controls/timer), Instructor Dashboard (Question Bank/Builder/Gradebook).
-
- **3.5.2 Hardware**: Host Server (min 2 GHz CPU, 4 GB RAM); Client Devices (PC/tablet) with keyboard/mouse/touch.
-
- **3.5.3 Software**: DB (PostgreSQL/MySQL via SQL/ORM); OS/Runtime (Linux/Windows, Node.js/Python).
-
- **3.5.4 Communication**: HTTPS encryption for RESTful HTTP GET/POST endpoints.
-
- **Section 3.1: Functional Requirements (FR16 – FR20)**

- **FR16 (Results UI)**: Post-quiz score, %, time, itemized feedback. *Primary*: Redirects to results. *Alt*: If hidden by instructor, shows score only.
-
- **FR17 (Student Gradebook)**: Tab for quiz history. *Primary*: Renders past quiz score table. *Alt*: Displays "No completed assessments found".
-
- **FR18 (Class Summary)**: Instructor analytics. *Primary*: Shows avg/max/min metrics + student score table. *Alt*: Dynamic search filter by name.
-
- **FR19 (CSV Export)**: Download results. *Primary*: Generates .csv file download. *Alt*: Disabled button with tooltip if 0 submissions.
-
- **FR20 (Email Confirmation)**: Automated receipt. *Primary*: Dispatches email with score/timestamp. *Alt*: Mail failure logs error without losing DB submission.
-
- **Section 3.2.2: Use Case Templates (UC16 – UC20)**
- **UC16 (View Results)**: *Actors*: Student | *Pre*: Submitted | *Flow*: Select quiz $\rightarrow$ Fetch record $\rightarrow$ Render score breakdown (or total only if hidden).
-
- **UC17 (Access Gradebook)**: *Actors*: Student | *Pre*: Authenticated | *Flow*: Click "My Grades" $\rightarrow$ Query DB by Student ID $\rightarrow$ Render history table.
-
- **UC18 (Class Summary)**: *Actors*: Instructor | *Pre*: Quiz selected | *Flow*: Open quiz $\rightarrow$ Click "Class Results" $\rightarrow$ Calculate stats $\rightarrow$ Render analytics dashboard.
-
- **UC19 (Export CSV)**: *Actors*: Instructor | *Pre*: Submissions exist | *Flow*: Open summary $\rightarrow$ Click "Export CSV" $\rightarrow$ Compile CSV payload $\rightarrow$ Trigger browser download.

-
- **UC20 (Email Confirmation)**: *Actors*: System | *Pre*: Submission saved | *Flow*: Quiz submitted $\rightarrow$ Generate payload $\rightarrow$ Dispatch email via SMTP.
-
- **Section 4.2: Appendix B — Glossary**
- **Auto-Grading**: Automated response evaluation against answer keys without human intervention.
-
- **CSV**: Plain text tabular spreadsheet file format.
-
- **HTTPS**: Encrypted web communication protocol.
-
- **MCQ**: Multiple Choice Question format.
-
- **MVC**: Model-View-Controller architectural pattern (Model = Data, View = UI, Controller = Logic).
-